

Project Interim Report

On

Customer Feedback Report (Using Core Java & OOP Concepts)

RU-100-01-00012

Object Oriented Programming with Java

SESSION: 2025-26



Bachelor of Technology in Computer Science & Engineering in
association with Google

Name :- **Ranjeet Kumar**

Erp :- **RU-25-11752**

Under the Guidance of **Mr. Ashish Shivhare**

**SCHOOL OF COMPUTER SCIENCE AND
ENGINEERING RUNGTA INTERNATIONAL SKILLS
UNIVERSITY**

BHILAI, CHHATTISGARH

Abstract

The Customer Feedback Analysis System is a Java-based desktop application developed using Object-Oriented Programming (OOP) principles. It is designed to help small businesses efficiently collect, store, and analyze customer feedback in a structured and automated manner. In today's competitive market, understanding customer sentiment is critical for business growth; however, many small enterprises rely on manual, paper-based processes that are error-prone and time-consuming. This system addresses that gap by providing a digital platform where feedback records can be added, managed, and evaluated based on ratings. The application computes average ratings, identifies trends, and displays meaningful results to the user through a menu-driven console interface. By replacing manual methods with a systematic OOP-based approach, the system reduces administrative effort and enables better decision-making. It is implemented using core Java with classes, arrays, and methods, making it lightweight, easy to understand, and extensible for future enhancements such as database connectivity and graphical user interfaces.

Introduction

In the modern business landscape, customer feedback plays a vital role in shaping products, services, and overall customer experience. Organizations that actively collect and analyze feedback gain a significant competitive advantage by understanding what their customers value and where improvements are needed. The Customer Feedback Analysis System is a console-based Java application that provides businesses with an efficient and organized way to manage this process.

The system is built entirely using core Java and follows Object-Oriented Programming paradigms including encapsulation, abstraction, and modularity. It allows users to add new feedback entries, assign numerical ratings, store data in memory using arrays, analyze the collected ratings to compute averages and identify high or low satisfaction areas, and display comprehensive results on the terminal. The application operates through an interactive menu that guides the user through each available operation. It is designed to be simple enough for non-technical users while demonstrating strong programming practices.

Key features of the system include:

- Add and store customer feedback with name, comment, and rating
- Retrieve and display all stored feedback records
- Analyze ratings to compute average and categorize satisfaction levels
- Clean menu-driven console interface for ease of use
- Modular design allowing easy future extension

Objectives

- Manage feedback records — allow users to add, store, and retrieve customer feedback entries systematically
- Track issued ratings — record and monitor numerical ratings provided by customers for analysis

Reduce manual work — replace paper-based or spreadsheet processes with an automated digital system

Apply OOP concepts — demonstrate encapsulation, abstraction, and modular class design using Java

Scope

This project is suitable for small businesses and can be extended with database integration in future. The current version supports in-memory data handling using Java arrays and provides a terminal based user interface. It is particularly useful for retail outlets, service centers, restaurants, and similar establishments that require a simple feedback tracking mechanism without the overhead of complex enterprise software. In future versions, the system scope can be broadened to include persistent storage through JDBC database connectivity, a graphical user interface using JavaFX or Swing, web based access for remote feedback submission, and integration with data visualization tools for advanced analytics.

System Requirements

Hardware: Computer with 4GB RAM

Software: Java JDK, IDE (VS Code / Eclipse / Edit Plus)

Methodology

The Customer Feedback Analysis System follows a straightforward, step-by-step workflow driven by a console menu. The methodology is described below:

Step 1 – Add Feedback:

The user selects the 'Add Feedback' option from the menu. The system prompts for the customer's name, a textual comment, and a numerical rating (typically on a scale of 1 to 5). This data is stored in an array of Feedback objects maintained in memory.

Step 2 – Store Data:

Feedback entries are stored using an array within the FeedbackManager class. Each entry is encapsulated as a Feedback object containing the relevant fields. The array index tracks the total number of entries added.

Step 3 – Analyze Rating:

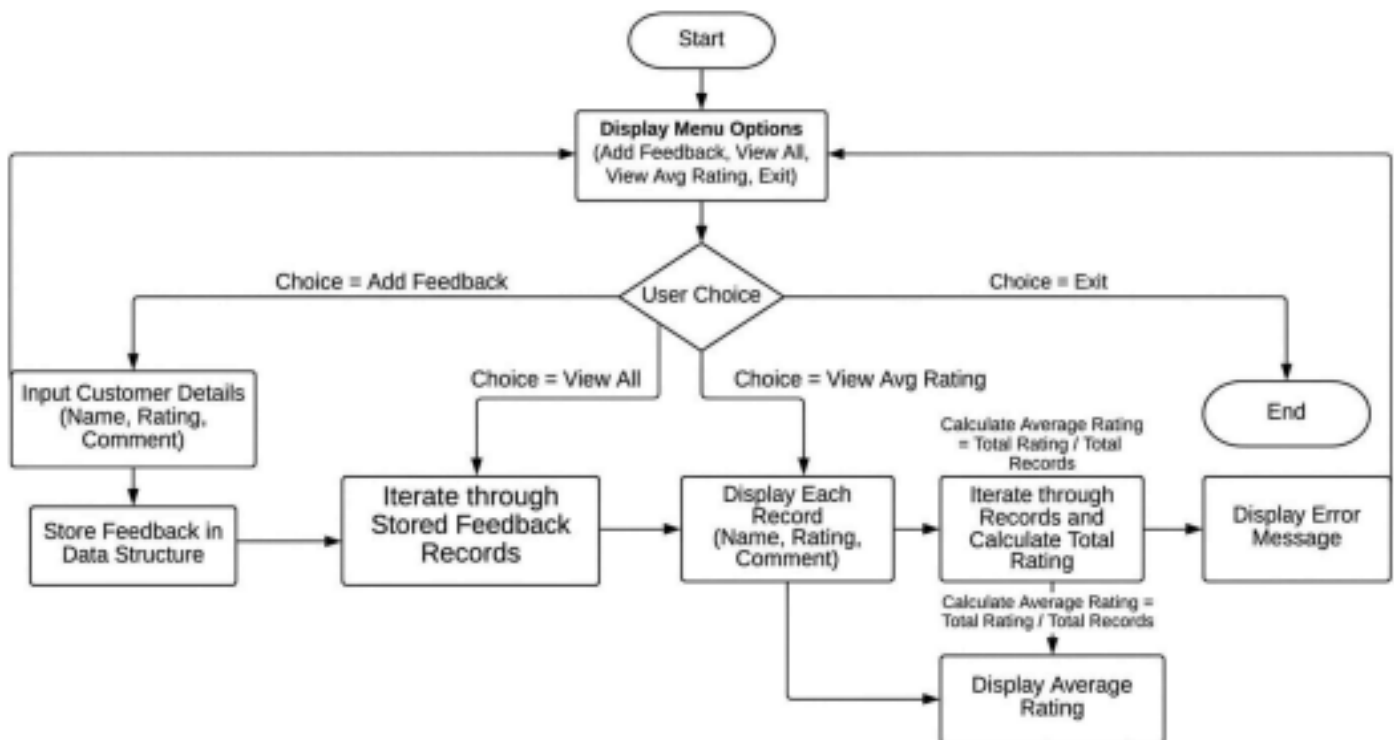
When the user selects the analysis option, the system iterates over all stored feedback records, sums the ratings, and computes the average. It also identifies entries with ratings below a threshold (e.g., 3) to flag low satisfaction instances for attention.

Step 4 – Display Result:

The results are formatted and printed to the console, showing each feedback record, the computed average rating, and a summary classification (e.g., Excellent, Good, or Needs Improvement) based on the average score.

Flowchart

The following text-based flowchart represents the overall execution flow of the system:



Class Diagram

The system is composed of three primary classes. Their structure is described below:

- customerName : String

- comment : String

```

- rating : int

+ Feedback(name, comment, rating)

+ getCustomerName() : String

+ getComment() : String

+ getRating() : int

+ displayFeedback() : void


- feedbackArray[] : Feedback // Array of feedback
objects - count : int // Tracks number of entries

- MAX_SIZE : int = 100


+ FeedbackManager()

+ addFeedback(Feedback f) : void

+ viewAllFeedback() : void

+ analyzeRatings() : void


(Entry point – no instance variables)


+ main(args[] : String) : void // Menu-driven driver

```

Implementation

The system is implemented across three Java classes, each serving a distinct role:

Feedback Class:

This class represents a single customer feedback entry. It encapsulates three private fields: `customerName` (a `String` storing the customer's name), `comment` (a `String` holding the textual feedback), and `rating` (an integer between 1 and 5). The constructor accepts all three values at the time of object creation. Getter methods expose each field, and a `displayFeedback()` method prints a formatted version of the entry to the console. This class demonstrates the OOP principle of encapsulation.

FeedbackManager Class:

This class manages the collection of all `Feedback` objects. It declares a fixed-size array of type `Feedback` with a maximum capacity of 100 entries and maintains a counter variable to track current usage. The `addFeedback()` method stores a new `Feedback` object at the next available array index and increments the counter. The `viewAllFeedback()` method iterates through the array up to the current count and calls `displayFeedback()` on each object. The `analyzeRatings()` method computes the sum of all ratings, divides by the count to find the average, and prints a summary along with a satisfaction category. Low-rated entries (below 3) are also flagged individually.

Main Class:

This is the entry point of the application. It initializes a `FeedbackManager` object and runs an infinite loop presenting the main menu. Using a switch-case structure, it reads the user's integer choice and delegates to the appropriate method. Input is handled through the `Scanner` class. The loop exits cleanly when the user selects option 4 (Exit).

Sample Input/Output

The following is a realistic terminal-style example of the system in operation:

```
=====
      CUSTOMER FEEDBACK ANALYSIS SYSTEM
=====

1. Add Feedback

2. View All Feedback

3. Analyze Ratings
```

4. Exit

Enter choice: 1

Enter Customer Name : Priya Sharma

Enter Comment : The service was prompt and staff was helpful. Enter

Rating (1-5) : 5

Feedback added successfully!

Enter choice: 1

Enter Customer Name : Rohit Verma

Enter Comment : Average experience, needs better packaging. Enter

Rating (1-5) : 3

Feedback added successfully!

Enter choice: 1

Enter Customer Name : Anita Joshi

Enter Comment : Very disappointed with delayed delivery. Enter

Rating (1-5) : 2

Feedback added successfully!

Enter choice: 2

- Feedback #1

Customer : Priya Sharma

Comment : The service was prompt and staff was

helpful. Rating : 5 / 5

- Feedback #2

Customer : Rohit Verma

Comment : Average experience, needs better
packaging. Rating : 3 / 5

- Feedback #3

Customer : Anita Joshi

Comment : Very disappointed with delayed delivery.

Rating : 2 / 5

Enter choice: 3

=====

= ANALYSIS REPORT

=====

= Total Feedback Received : 3

Total Rating Score : 10

Average Rating : 3.33 / 5
Overall Satisfaction : Good

Low-Rated Feedback (Rating < 3):

- Anita Joshi (Rating: 2) - Very disappointed with delayed
delivery. =====

Enter choice: 4

Thank you for using the Feedback System. Goodbye!

Future Enhancements

The current system provides a functional foundation that can be significantly extended in future iterations:

Database Integration — Connect the system to a relational database (MySQL or PostgreSQL) using JDBC to enable persistent storage, allowing feedback records to survive across application restarts.

Graphical User Interface (GUI) — Replace the console interface with a modern GUI built using JavaFX or Swing, improving usability for non-technical users.

AI-Powered Sentiment Analysis — Incorporate Natural Language Processing (NLP) or machine learning models to automatically classify the tone of textual comments as positive, neutral, or negative.

Web-Based Access — Convert the application into a web platform using Spring Boot and REST APIs, enabling customers to submit feedback remotely via a browser or mobile device.

Data Visualization — Integrate charting libraries to generate graphical reports such as bar charts of average ratings, trend lines over time, and pie charts of satisfaction distribution.

Email Notifications — Automatically alert business managers via email when the average rating drops below a defined threshold, enabling timely corrective action.

Export Functionality — Allow users to export feedback reports as PDF or Excel files for record keeping and presentation purposes.

Gantt Chart

1 Requirement Analysis Week 1 Week 1 2 System Design &

Class Planning Week 2 Week 2 3 Implementation – Feedback Module Week 3 Week 3 4

Implementation – Rating & Analysis Week 4 Week 4 5 Testing & Debugging Week 5 Week 5 6

Documentation & Report Writing Week 6 Week 6

Conclusion

The Customer Feedback Analysis System successfully demonstrates the practical application of Object-Oriented Programming principles in solving a real-world business problem. Through the use of encapsulation, the Feedback class secures data integrity by restricting direct access to its fields. The FeedbackManager class exemplifies abstraction and modularity by grouping all data management operations under a single, cohesive unit. The array-based storage mechanism reflects an understanding of fundamental data structures within the Java environment.

The system fulfills all defined objectives: it manages feedback records efficiently, tracks customer ratings, significantly reduces manual effort compared to traditional methods, and clearly illustrates the advantages of an OOP-driven design. The menu-driven console interface ensures that the application is accessible and intuitive despite its simplicity.

This project serves as a solid base for further development. With enhancements such as database integration, AI-based sentiment analysis, and a web-accessible front end, the system has the potential to grow into a robust enterprise-grade feedback management platform. Overall, the Customer Feedback Analysis System is a meaningful and technically sound project that demonstrates both programming competence and practical problem-solving ability.

References

- [1] H. Schildt, Java: The Complete Reference, 12th ed. New York: McGraw-Hill Education, 2021.
- [2] Oracle Corporation, "Java SE Documentation," Oracle. [Online].
Available: <https://docs.oracle.com/en/java/>
- [3] R. Singh and P. Mehta, "Automated Feedback Collection Systems in Retail Environments," in Proc. IEEE International Conference on Computing and Communication Systems (I3CS), Shillong, India, 2022, pp. 210–215.
- [4] A. Sharma and V. Gupta, "Customer Satisfaction Analysis Using Object-Oriented Design Patterns," International Journal of Computer Applications, vol. 183, no. 12, pp. 34–39, 2021.